# Introduction to Image Processing

## Practical Work No. 2

---

**Student:** Ibrahim Ege Bilgic

**Student ID No.:** 231ADB284

**Link to the programming code:** https://github.com/ibrahimegebilgic/practical2-imageprocessing/blob/main/Practical_2.ipynb

**Variant:** 4 (Logarithmic correction, Histogram linear correction (min-max))

---

In [1]:
```
import os
import numpy as np
import cv2
import matplotlib.pyplot as plt
from google.colab import files
```

### Upload 3 color images

- Underexposed (dark)
- Overexposed (bright)
- Grayish (low contrast)

After upload, rename the files on your computer so their names include one of these keywords: `under`, `over`, `gray`.

In [2]:
```
uploaded = files.upload()
paths = list(uploaded.keys())
paths
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving alpine-hare-edwin-godinho.webp to alpine-hare-edwin-godinho.webp
```

Out[2]:
```
['alpine-hare-edwin-godinho.webp']
```

In [3]:
```
def to_rgb_uint8(bgr):
    rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
    if rgb.dtype != np.uint8:
        rgb = np.clip(rgb, 0, 255).astype(np.uint8)
    return rgb

def log_correction_rgb(img_rgb, a=12.0):
    x = img_rgb.astype(np.float32) / 255.0
```

```
    y = np.empty_like(x)
    denom = np.log(1.0 + a)
    y[..., 0] = np.log(1.0 + a * x[..., 0]) / denom
    y[..., 1] = np.log(1.0 + a * x[..., 1]) / denom
    y[..., 2] = np.log(1.0 + a * x[..., 2]) / denom
    return np.clip(y * 255.0, 0, 255).astype(np.uint8)

def minmax_linear_rgb(img_rgb):
    x = img_rgb.astype(np.float32)
    y = np.empty_like(x)
    for c in range(3):
        ch = x[..., c]
        mn = float(np.min(ch))
        mx = float(np.max(ch))
        if mx == mn:
            y[..., c] = ch
        else:
            y[..., c] = (ch - mn) * (255.0 / (mx - mn))
    return np.clip(y, 0, 255).astype(np.uint8)
```

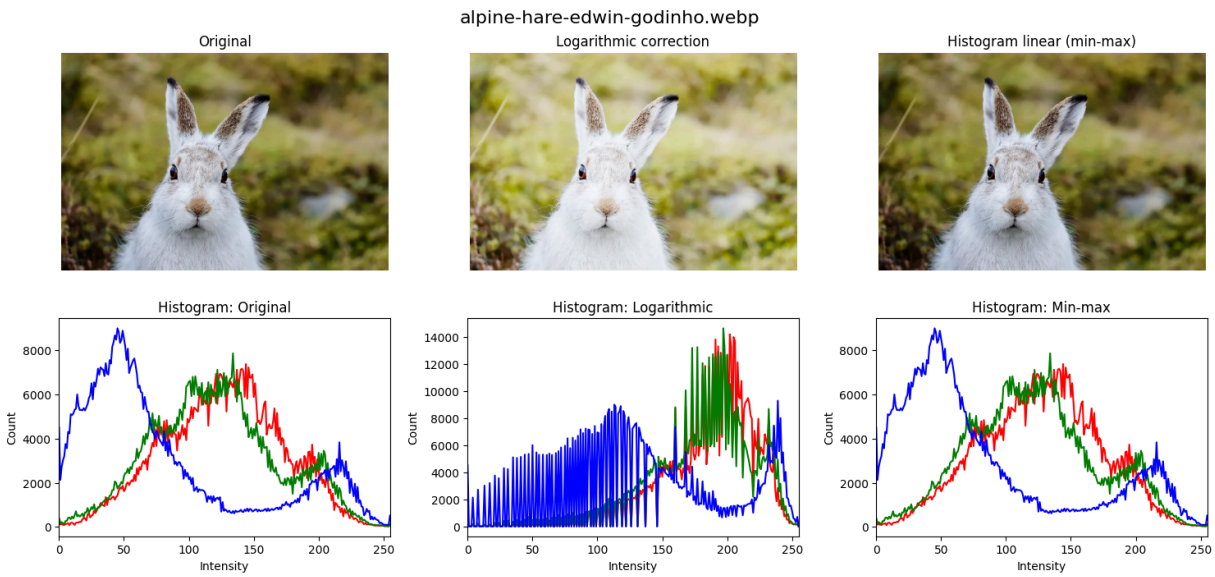
In [4]:

```
def plot_rgb_hist(ax, img_rgb, title):
    ax.set_title(title)
    ax.set_xlim(0, 255)
    for i, col in enumerate(["r", "g", "b"]):
        h = cv2.calcHist([img_rgb], [i], None, [256], [0, 256]).ravel()
        ax.plot(h, color=col)
    ax.set_xlabel("Intensity")
    ax.set_ylabel("Count")

def show_results(name, orig, out1, out2):
    plt.figure(figsize=(15, 10))
    plt.suptitle(name, fontsize=16)
    ax1 = plt.subplot(3, 3, 1)
    ax1.imshow(orig)
    ax1.set_title("Original")
    ax1.axis("off")
    ax2 = plt.subplot(3, 3, 2)
    ax2.imshow(out1)
    ax2.set_title("Logarithmic correction")
    ax2.axis("off")
    ax3 = plt.subplot(3, 3, 3)
    ax3.imshow(out2)
    ax3.set_title("Histogram linear (min-max)")
    ax3.axis("off")
    ax4 = plt.subplot(3, 3, 4)
    plot_rgb_hist(ax4, orig, "Histogram: Original")
    ax5 = plt.subplot(3, 3, 5)
    plot_rgb_hist(ax5, out1, "Histogram: Logarithmic")
    ax6 = plt.subplot(3, 3, 6)
    plot_rgb_hist(ax6, out2, "Histogram: Min-max")
    plt.tight_layout()
    plt.show()
```

In [5]:

```
LOG_A = 12.0
```

```python
def label_from_name(fname):
    f = fname.lower()
    if "under" in f:
        return "Underexposed"
    if "over" in f:
        return "Overexposed"
    if "gray" in f or "grey" in f or "low" in f:
        return "Grayish (low contrast)"
    return fname

for p in paths:
    bgr = cv2.imread(p)
    if bgr is None:
        print("Could not read:", p)
        continue
    orig = to_rgb_uint8(bgr)
    out_log = log_correction_rgb(orig, a=LOG_A)
    out_mm = minmax_linear_rgb(orig)
    show_results(label_from_name(p), orig, out_log, out_mm)
```




In [6]:
```python
out_dir = "pw2_outputs"
os.makedirs(out_dir, exist_ok=True)

for p in paths:
    bgr = cv2.imread(p)
    if bgr is None:
        continue
    orig = to_rgb_uint8(bgr)
    out_log = log_correction_rgb(orig, a=LOG_A)
    out_mm = minmax_linear_rgb(orig)
    base = os.path.splitext(os.path.basename(p))[0]
    cv2.imwrite(os.path.join(out_dir, base + "_log.png"), cv2.cvtColor(out_log, cv2
    cv2.imwrite(os.path.join(out_dir, base + "_minmax.png"), cv2.cvtColor(out_mm, c

sorted(os.listdir(out_dir))
```

Out[6]: ['alpine-hare-edwin-godinho_log.png', 'alpine-hare-edwin-godinho_minmax.png']

In [7]:
```
!pip install nbconvert
```

```
Requirement already satisfied: nbconvert in /usr/local/lib/python3.12/dist-packages
(7.17.0)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-pack
ages (from nbconvert) (4.13.5)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-packa
ges (from bleach[css]!=5.0.0->nbconvert) (6.3.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-packages
(from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in /usr/local/lib/python3.12/dist-package
s (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in /usr/local/lib/python3.12/dist-p
ackages (from nbconvert) (5.9.1)
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.12/dist
-packages (from nbconvert) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in /usr/local/lib/python3.12/dist-pac
kages (from nbconvert) (3.0.3)
Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.12/dist-p
ackages (from nbconvert) (3.2.0)
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.12/dist-pac
kages (from nbconvert) (0.10.4)
Requirement already satisfied: nbformat>=5.7 in /usr/local/lib/python3.12/dist-packa
ges (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages
(from nbconvert) (26.0)
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.12/dis
t-packages (from nbconvert) (1.5.1)
Requirement already satisfied: pygments>=2.4.1 in /usr/local/lib/python3.12/dist-pac
kages (from nbconvert) (2.19.2)
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-pack
ages (from nbconvert) (5.7.1)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packag
es (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.12/dis
t-packages (from bleach[css]!=5.0.0->nbconvert) (1.4.0)
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-p
ackages (from jupyter-core>=4.7->nbconvert) (4.9.2)
Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/python3.12/d
ist-packages (from nbclient>=0.5.0->nbconvert) (7.4.9)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dis
t-packages (from nbformat>=5.7->nbconvert) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-pac
kages (from nbformat>=5.7->nbconvert) (4.26.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packa
ges (from beautifulsoup4->nbconvert) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.1
2/dist-packages (from beautifulsoup4->nbconvert) (4.15.0)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packa
ges (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.4.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/li
b/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (2025.9.
1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist
-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-pac
kages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.30.0)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-package
```

```
s (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (0.4)
Requirement already satisfied: nest-asyncio>=1.5.4 in /usr/local/lib/python3.12/dist
-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/d
ist-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: pyzmq>=23.0 in /usr/local/lib/python3.12/dist-package
s (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (26.2.1)
Requirement already satisfied: tornado>=6.2 in /usr/local/lib/python3.12/dist-packag
es (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.5.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages
(from python-dateutil>=2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert)
(1.17.0)
```

In [11]:
```python
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.
mount("/content/drive", force_remount=True).
```

In [12]:
```
!jupyter nbconvert "/content/drive/MyDrive/Colab Notebooks/Practical_2.ipynb" \
  --to html \
  --embed-images
```

```
[NbConvertApp] Converting notebook /content/drive/MyDrive/Colab Notebooks/Practical_
2.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 1 image(s).
[NbConvertApp] Writing 979392 bytes to /content/drive/MyDrive/Colab Notebooks/Practi
cal_2.html
```

In [13]:
```python
from google.colab import files
files.download("/content/drive/MyDrive/Colab Notebooks/Practical_2.html")
```

# Comparison

Logarithmic correction makes dark parts brighter. It helps a lot when the image is too dark. The shadows become clearer and you can see more details. The histogram also moves to the right, mainly in the low values. Min–max correction is different. It stretches the values from the lowest to the highest. So the contrast becomes stronger in the whole image. The histogram becomes wider and fills more of the 0–255 range.

For the underexposed image, I think logarithmic correction keeps details better. It opens the shadows and does not make everything too harsh. Min–max can make some areas too black, so small details can disappear. For the low-contrast gray image, min–max works better. It makes the image look sharper and more clear. Log correction improves it, but not as much. For the overexposed image, both methods are limited. Very bright parts already lost information, so the details cannot really come back.

Log correction is good because it is simple and looks natural. It is very useful for dark images. But it can also show more noise in shadow areas. Min–max is good because it

increases contrast a lot and uses the full range. It is great for flat images. But sometimes it looks too strong, and it can lose details in extreme dark or bright parts.

# Conclusions

In this practical work, I improved three different images using logarithmic correction and histogram linear correction (min–max). Logarithmic correction made the dark (underexposed) image brighter and revealed more details in the shadows. However, it can also increase noise in very dark areas. Min–max correction increased the contrast strongly because it stretches pixel values to the full range (0–255). For the grayish (low contrast) image, min–max worked very well and the picture looked clearer. For the overexposed image, both methods had limits, because bright areas were already clipped and lost information. From the histograms, I saw that min–max spreads the values more, while log correction changes mainly the darker part of the histogram. Overall, I would choose log correction for underexposed images and min–max for low-contrast gray images, but I must be careful with overexposed photos.